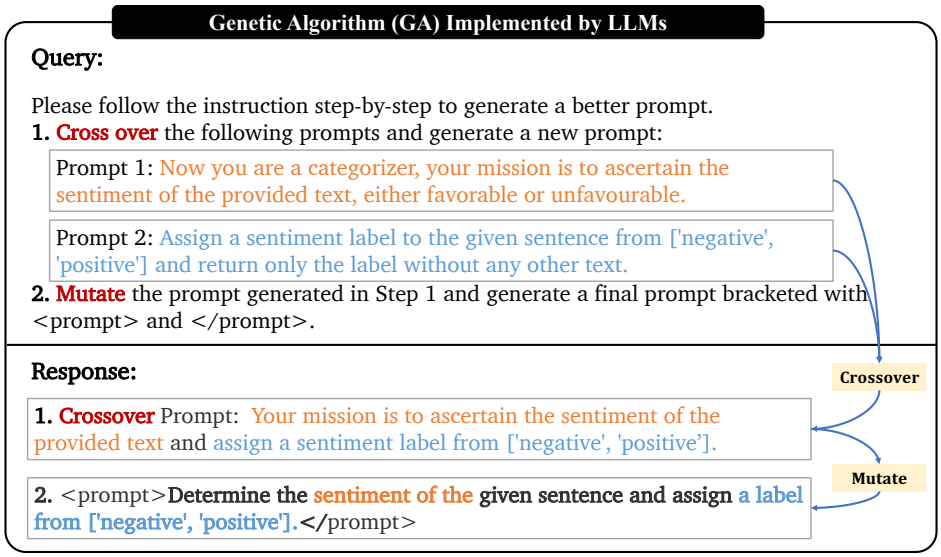

Figure 1: GA process implemented by LLMs (Evo$(\cdot)$ in Algorithm 1). In Step 1, LLMs perform *crossover* on the given two prompts (words in orange and blue are inherited from Prompt 1 and Prompt 2, respectively). In Step 2, LLMs perform *mutation* on the prompt.

In order to implement EvoPrompt in practice, it is necessary to instantiate it with a specific algorithm of EAs. There are various types of EAs, and in this paper, we consider two widely used algorithms, including Genetic Algorithm (GA) (Holland, 1975) and Differential Evolution (DE) (Storn & Price, 1997). GA is among the most highly regarded evolutionary algorithms (Holland, 1975; 1992; Mitchell, 1998; Mirjalili et al., 2020) and DE has emerged as one of the most widely utilized algorithms for complex optimization challenges since its inception (Storn & Price, 1997; Price, 2013; Das & Suganthan, 2010; Pant et al., 2020). In the following, we will first outline the proposed EvoPrompt, and then instantiate EvoPrompt with GA and DE respectively.

3.1 Framework of EvoPrompt

EAs typically start with an initial population of N solutions (prompts in our setting), then iteratively generate new solutions using evolutionary operators (e.g., mutation and crossover) on the current population and update it based on a fitness function. Following typical EAs, EvoPrompt mainly contains three steps:

- **Initial population**: Contrary to most existing automatic prompt methods that neglect priori human knowledge, we apply available manual prompts as the initial population to leverage the wisdom of humans. Besides, EAs typically start from random solutions, resulting in a diverse population and avoiding being trapped in a local optimum. Accordingly, we also introduce some prompts generated by LLMs (Zhou et al., 2022) into the initial population.
- **Evolution**: In each iteration, EvoPrompt uses LLMs as evolutionary operators to generate a new prompt based on several parent prompts selected from the current population. To accomplish this, we design steps of the *mutation* and *crossover* operators for each specific type of EAs, along with corresponding instructions to guide the LLMs in generating new prompts based on these steps.
- **Update**: We evaluate the generated candidate prompts on a development set and retain those with superior performance, similar to the survival of the fittest in nature. The specific updating strategy may vary depending on the type of EAs used.

The algorithm stops when the number of iterations reaches a predefined value. The details of EvoPrompt are outlined in Algorithm 1. When instantiating EvoPrompt with a specific algorithm of EAs, the evolutionary processes need to be adjusted, and the key challenge is to design the evolutionary operators on discrete prompts.

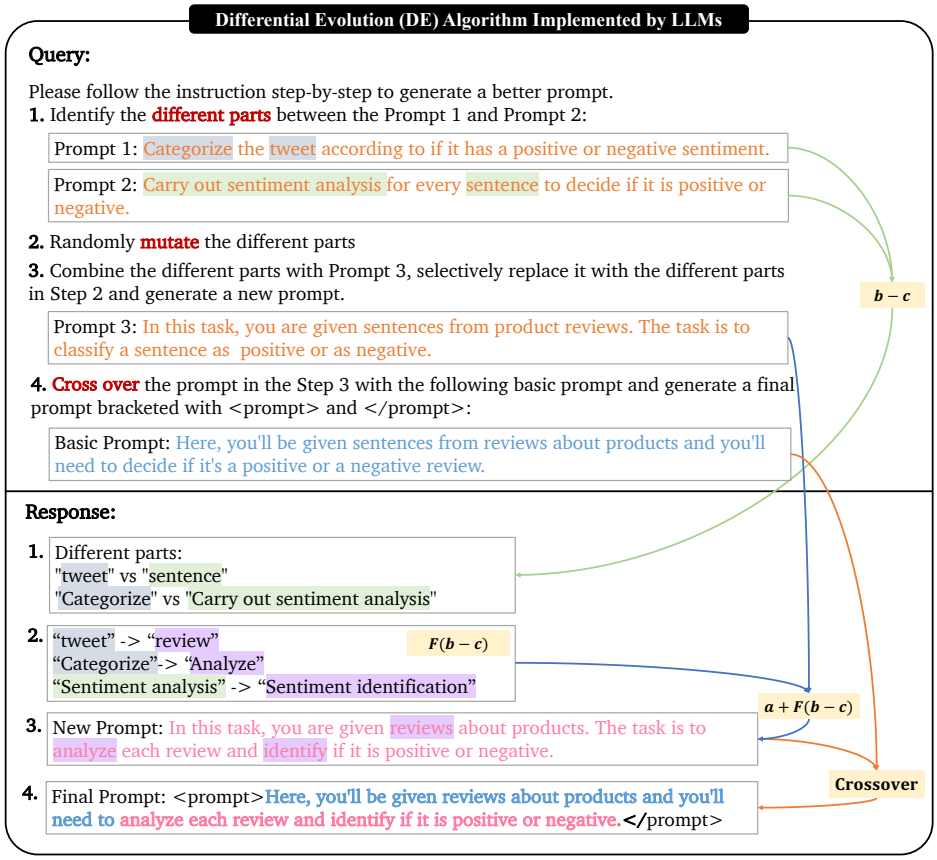

Figure 2: DE process implemented by LLMs (Evo$(\cdot)$ in Algorithm 1). In Step 1, LLMs find the different parts (words in ■ and ■) between Prompt 1 and Prompt 2 ($\mathbf{b}-\mathbf{c}$ in typical DE). In Step 2, LLMs perform *mutation* (words in ■) on them (imitation of $\mathbf{F}(\mathbf{b}-\mathbf{c})$). Next, LLMs incorporate the current best prompt as Prompt 3 with the mutated results in Step 2, to generate a new prompt (counterpart of $\mathbf{a}+\mathbf{F}(\mathbf{b}-\mathbf{c})$ in DE). Finally, LLMs perform *crossover* upon the current basic prompt p_i and the generated prompt in Step 3. See Figure 5 in Appendix B.2 for the complete response.

3.2 Instantiation with Genetic Algorithm

Selection In GA, parent solutions are conventionally selected using the roulette wheel selection method, guided by their fitness values (Lipowski & Lipowska, 2012). Analogously, we employ the roulette wheel selection to choose two parent prompts from the current population, based on their performance scores obtained on the development sets. Let s_i denote the performance score of the i-th prompt within a population containing N prompts. The probability of selecting the i-th prompt as a parent can be expressed as $p_i = s_i / \sum_{j=1}^{N} s_j$.

Evolution Conforming to the GA framework, we generate a new candidate prompt via two steps: **1)** Crossover is performed between the parent prompts to produce a new offspring prompt that inherits characteristics from both parents; **2)** Mutation is applied to the offspring prompt, introducing random alterations to certain elements. We formalize this two-stage operation into algorithmic instructions for guiding LLMs to implement Evo$(\cdot)$ in Algorithm 1. The entire process is illustrated in Figure 1.

Update We employ a straightforward selection strategy for updating the population: at each iteration, EvoPrompt produces N new prompts, which are merged with the existing population of N prompts. Subsequently, the top N prompts, based on their scores, are retained to form the updated population. Accordingly, the overall quality of the population undergoes continuous enhancement, culminating in the selection of the best one within the final population as the optimal prompt.

3.3 INSTANTIATION WITH DIFFERENTIAL EVOLUTION

Here, we begin with some preliminary knowledge of DE. Unlike GA, the solutions of DE are represented by numerical vectors. Each vector within the population is sequentially selected as a base vector, denoted as $\mathbf{x}$, which subsequently undergoes mutation and crossover. During mutation, a mutated solution $\mathbf{y}$ is generated from a randomly selected solution $\mathbf{a}$ from the current population. The mutation is achieved by adding a scaled difference between two distinct, randomly selected solutions $\mathbf{b}$ and $\mathbf{c}$ to $\mathbf{a}$, i.e., $\mathbf{y} = \mathbf{a} + F(\mathbf{b} - \mathbf{c})$, where F is the scaled parameter.

Crossover is to generate a trial solution $\mathbf{x}' = [x'_1, ..., x'_n]$ by choosing each parameter in the vector from either the basic solution $\mathbf{x}$ or the mutated solution $\mathbf{y}$. Then, $\mathbf{x}$ is replaced with $\mathbf{x}'$ if $\mathbf{x}'$ is better than $\mathbf{x}$. Within step-by-step evolution, DE ends with a population of high quality. A modified version of DE uses the current best solution as vector $\mathbf{a}$ to exploit information from the best one.

Evolution The evolutionary process of DE can be decoupled into three steps: 1) $F(\mathbf{b} - \mathbf{c})$; 2) $\mathbf{y} = \mathbf{a} + F(\mathbf{b} - \mathbf{c})$; 3) Crossover of $\mathbf{x}$ and $\mathbf{y}$. In EVOPROMPT based on DE, we follow the three steps to design the evolutionary process, as well as the corresponding instructions for LLMs to generate a new prompt based on these steps as illustrated in Figure 2:

- Inspired by the differential vector in DE, we consider mutating only the different parts of two randomly selected prompts in the current population (Step 1 and Step 2 in Figure 2). The prompts in the current population are considered the current best ones. Accordingly, the shared components of two prompts tend to have a positive impact on the performance, and thus need to be preserved.
- A variant of DE employs the current best vector during the mutation process, where a mutated vector is generated by adding the scale of the differential vector to the current best vector. Building upon this idea, we generate a mutated prompt by selectively replacing parts of the current best one with the mutated different parts for combination. (Step 3 in Figure 2).
- Crossover replaces certain components of a basic prompt (i.e., a candidate of the current population) with segments from the mutated prompt. This operation combines the features of two different prompts, potentially creating a new and improved solution (Step 4 in Figure 2).

Update Following the standard DE, each prompt p_i in the current population is chosen as a basic prompt in turn to generate a corresponding new prompt p'_i using the instruction in Figure 2. Then, the prompt with a higher score, either p_i or p'_i, is retained. Accordingly, the population size remains constant while the overall quality of the population is enhanced.

4 EXPERIMENTS

4.1 IMPLEMENTATION DETAILS AND BASELINES

With GPT-3.5 performing evolutionary operators, we optimize prompts using EVOPROMPT for the open-source Alpaca-7b (Taori et al., 2023) and closed-source GPT-3.5 (text-davinci-003) (Brown et al., 2020). We pick the prompt with the highest score on the development set and report its score on the test set. Results reported on Alpaca are averaged over 3 random seeds and the standard deviation is provided, while for GPT-3.5, we report results of one seed due to budget limitation. In our evaluation, we compare EVOPROMPT against three categories of prompt-based approaches, detailed as follows:

- **Manual Instructions (MI)**: These serve as task-specific guidelines and are crafted based on established works, specifically referenced from Zhang et al. (2023b) for language understanding, Sanh et al. (2021) for summarization, and Zhang et al. (2023c) for text simplification.
- **PromptSource** (Bach et al., 2022) and **Natural Instructions (NI)** (Mishra et al., 2022b): These repositories aggregate human-composed prompts across a diverse range of datasets.
- **APE** (Zhou et al., 2022) and **APO** (Pryzant et al., 2023): APE employs an iterative Monte Carlo Search strategy, emphasizing on *exploration*. We reproduce it and initialize populations of equivalent sizes to that of EVOPROMPT. APO harnesses incorrectly predicted instances as "pseudo-gradient" to iteratively refine the original prompt, which emphasizes *exploitation*. We reproduce APO on binary classification tasks with the optimal manual prompt as the initial one.